

CHAPTER 16: Flat Clustering

★ clustering:

- **Unsupervised** ML method
- algorithm studies the data itself and discovers patterns automatically
- main goal is:
 - ✓ Items inside the same cluster should be **very similar**
 - ✓ Items in different clusters should be **very different**
- The **data distribution + similarity between items** determine which cluster something belongs to.

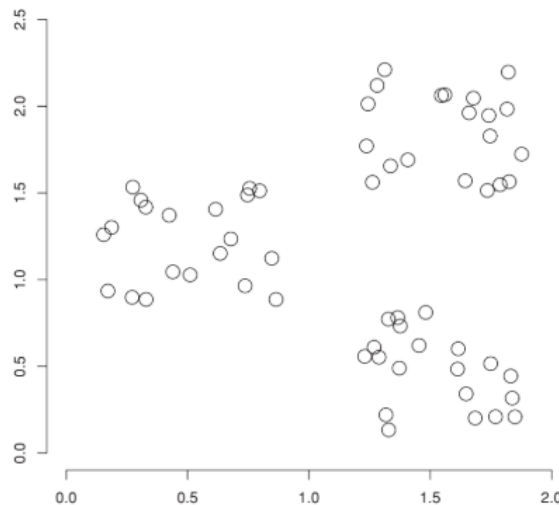


Figure 16.1 An example of a data set with a clear cluster structure.



Clustering vs Classification

- **Clustering:** clusters are **inferred from data without human input**
- **Classification:** classes are human defined and part of input to the learning algo

Feature	Clustering	Classification
Learning Type	Unsupervised	Supervised
Labels Provided?	✗ No	✓ Yes
Human Teacher?	✗ No	✓ Yes
Goal	Discover hidden groups	Predict predefined classes

Distance Measure in Clustering

distance measure → How similar or different two documents/data points are.

- Different distance measures produce different clusterings.
- distance measure can heavily influence the outcome of clustering.

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Types of Clustering

> two major types discussed here:

1. Flat Clustering

- Flat clustering creates clusters without relationships between them.

- All clusters exist at the same level.
- a flat set of clusters without any explicit structure that would relate clusters to each other.

2. Hierarchical Clustering

- Hierarchical clustering creates a **tree-like structure of clusters**.
 - Clusters are related to each other.
 - **Clusters within clusters**
-

Hard vs Soft Clustering

Hard Clustering

- Each document belongs to **exactly ONE cluster**.
- **No overlap** allowed.

Soft Clustering

- A document can **partially belong to multiple clusters**.
 - **Membership** is **fractional/probabilistic**.
 - **Fuzzy** membership in more than 1 cluster
-

[16.1] Clustering in IR

extremely useful in IR, especially in **search engines** and **document organization** systems.

Cluster Hypothesis

The **Cluster Hypothesis** is the main assumption behind using clustering in IR.

Statement

Documents in the same cluster behave similarly with respect to relevance to information needs.

> If **one document in a cluster** is **relevant to** a **query** → Then **other documents** in the **same cluster** are **also likely relevant**.

- All applications of clustering in IR are based (directly or indirectly) on the **cluster hypothesis**
- closely associated documents tend to be relevant to the same requests.

Relation to Contiguity Hypothesis

The text says **cluster hypothesis** is **similar** to the **contiguity hypothesis**.

Core Idea

Similar documents behave similarly regarding relevance.

So:  Similar documents → similar usefulness for the user.

Applications of Clustering in IR

application	What is clustered?	benefit	example
search result clustering	search results	more effective information presentation to user	Figure 16.2
scatter-gather	(subsets of) collection	alternative user interface: "search without typing"	Figure 16.3
collection clustering	collection	effective information presentation for exploratory browsing	McKeown et al. (2002), http://news.google.com
language modeling	collection	increased precision and/or recall	Liu and Croft (2004)
cluster-based retrieval	collection	higher efficiency: faster search	Salton (1971a)

The screenshot shows the Vivísimo search engine interface. At the top, there is a search bar with the text 'jaguar' and a dropdown menu set to 'the Web'. To the right of the search bar is a 'Search' button and links for 'Advanced Search' and 'Help'. Below the search bar, a status bar indicates 'Top 208 results of at least 20,373,974 retrieved for the query jaguar (Details)'. On the left side, there is a 'Clustered Results' panel. This panel lists various clusters related to the search term 'jaguar', including 'Cars (74)', 'Club (34)', 'Cat (23)', 'Animal (13)', 'Restoration (10)', 'Mac OS X (8)', 'Jaguar Model (8)', 'Request (5)', 'Mark Webber (6)', and 'Maya (5)'. The 'Cat' cluster is highlighted with a third arrow from the top. To the right of the clustered results, the top search results are listed. The first result is 'Jag-lovers - THE source for all JAGUAR information', which is a website for Jaguar enthusiasts. The second result is 'Jaguar Cars', which is a website for Jaguar cars. The third result is 'http://www.jaguar.com/', which is the official Jaguar website. The fourth result is 'Apple - Mac OS X', which is a website for Apple's Mac OS X operating system. The 'Cat' cluster in the left panel is highlighted with a yellow background, and a yellow box around the text 'Clustering of search results' in the caption points to this cluster.

Figure 16.2 Clustering of search results to improve recall. None of the top hits cover the animal sense of jaguar, but users can easily access it by clicking on the *Cat Cluster* in the *Clustered Results* panel on the left (third arrow from the top).

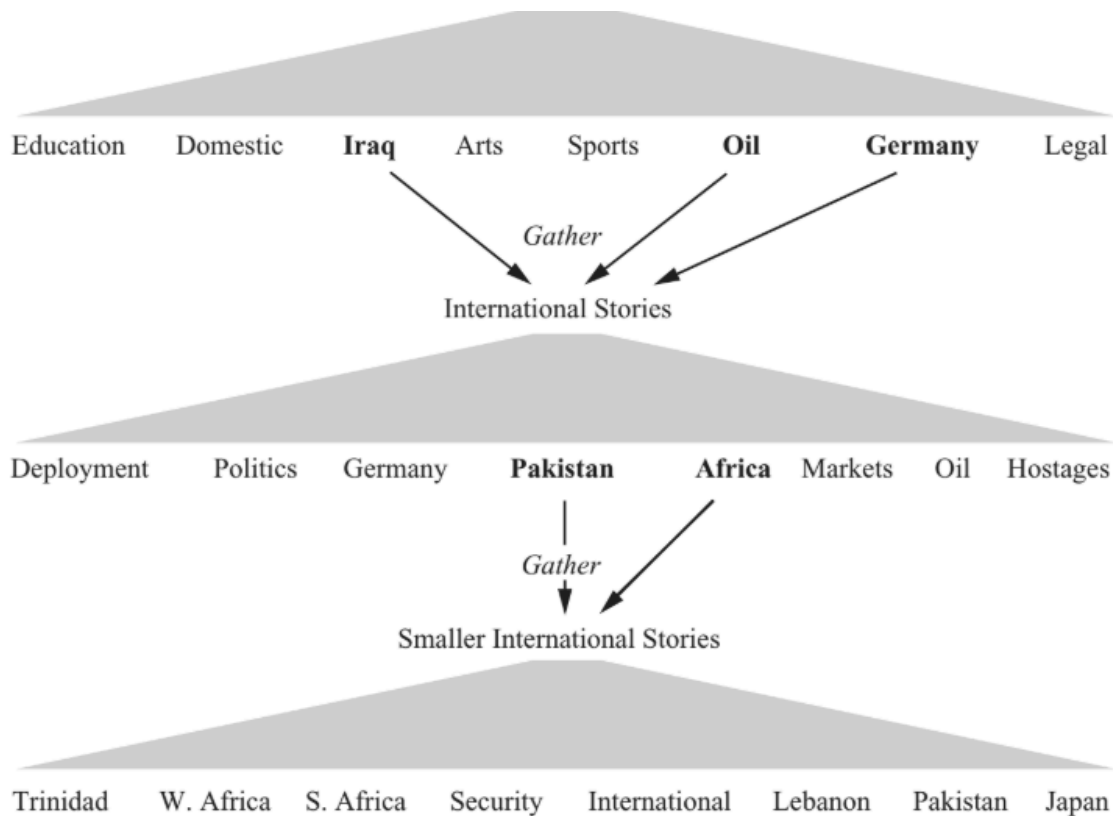


Figure 16.3 An example of a user session in scatter-gather. A collection of *New York Times* news stories is clustered ("scattered") into eight clusters (*top row*). The user manually *gathers* three of these into a smaller collection *International Stories* and performs another scattering operation. This process repeats until a small cluster with relevant documents is found (e.g., *Trinidad*).

1. Search Result Clustering [Vivísimo Search Engine]

 **What is clustered?** > Search results returned by a query.

 **Goal** > Improve the presentation of search results.

Problem with Traditional Search

Normally search engines show:

1 Result 1

2 Result 2

3 Result 3

...

Users scan line by line from top to bottom.

This becomes **difficult when**:

- **results are mixed**
 - **query has multiple meanings**
-

 **Example: "Jaguar"**

The word **Jaguar** has **multiple meanings**:

 Jaguar car

 Jaguar animal

 Apple Mac OS X Jaguar

A normal ranked list **mixes all these together**.

Search Result Clustering Solution

The search engine **groups results into clusters**.

Benefits

Users can **quickly jump to the category they want** instead of scanning hundreds of pages.

> **improves: User experience + Navigation + Recall of relevant documents**

Why Clustering Improves Recall

In Figure 16.2: Top results mostly discuss Jaguar cars 🚗.

Without clustering, users may never discover Jaguar animal results 🐆.

But **clustering creates a separate “Cat” cluster**, making animal-related results **easy to access**.

2. Scatter-Gather

> Scatter-gather is an **interactive browsing method** using clustering.

Two Main Steps

Step 1: Scatter

- The **collection is divided into clusters**.

Step 2: Gather

- The **user selects interesting clusters**.
- **Selected clusters** are **merged** together.
- Then **clustering is repeated again**.

Figure 16.3 Explanation

First Scatter

News articles are clustered into:

- Iraq
- Oil
- Germany

- Sports
- Arts
- Education
- Legal

The user selects:

- ☒ Iraq
- ☒ Oil
- ☒ Germany

These are **gathered into**: 📁 **International Stories**

Second Scatter

The smaller collection is **reclustered** into:

- Pakistan
- Africa
- Markets
- Oil
- Hostages

The **user chooses**:

- ☒ Pakistan
- ☒ Africa

These are **gathered again**.

Third Scatter

Now the clusters become more specific:

- Trinidad
- Lebanon
- Pakistan
- Japan
- Security

Eventually the **user finds the desired topic.**

Main Idea of Scatter-Gather

Instead of typing search keywords:  **Users navigate by browsing clusters.**

Benefits

Useful when users:

- **don't know exact keywords**
- want **exploratory browsing**
- want **topic discovery**

Problems

Automatically generated clusters are often:

-  messy
 -  poorly labeled
 -  less organized than manually designed directories
-

Comparison with Open Directory

DMOZ was a **manually built hierarchical web directory**.

Human-created categories were **cleaner** and **more organized**.

3. Collection Clustering

- The **entire collection is clustered beforehand**
- **Static hierarchical** clustering
- **Not influenced by user interactions**

Example

- **Google News** uses collection clustering.

News articles are **grouped into topics automatically**.

Example:

 Elections +  Sports +  International +  Economy

Why Clustering Works Well for News

Reading news is usually:

 not exact searching

 **browsing recent events**

Users **explore topics** rather than search precise keywords.

Dynamic Re-Clustering

News changes rapidly.

So clustering must be **recomputed frequently** to include:

 Breaking news,  New events,  Trending topics

4. Cluster-Based Retrieval

Use clusters directly to improve retrieval quality.

- Higher efficiency , faster search

Process

Step 1: Use **normal search to retrieve initial documents.**

Step 2: **Find clusters containing these relevant documents.**

Step 3: **Add more documents from the same clusters.**

Even if they don't directly match the query.

Example

Query: car

Initial results may contain documents with word: **car**

But other relevant docs may use:

- automobile
- vehicle
- sedan

Cluster membership helps retrieve these too.

✅ Benefit

This **improves**: 📌 **Recall**

More relevant documents are found.

Because related documents often exist in the same cluster.

🧠 5. Clustering in Language Modeling

> Language models suffer from **sparse data problems**.

Some **terms appear very rarely**.

📌 Traditional Solution

Use:

- document model + entire collection model

combined together.

❌ Problem with Whole Collection

The collection contains many unrelated documents.

This introduces irrelevant word probabilities.

🌟 Better Solution

Use the **document's cluster** instead of the entire collection.

Because documents inside the cluster are more similar.

✓ Result

More **accurate term probability estimation**.

Better search quality.

⚡ Clustering for Faster Search

- **Searching every document individually** can be **slow**.
- Especially in systems like: Latent Semantic Indexing
 - where inverted indexes may not work efficiently.

🧠 Solution Using Clusters

Instead of searching all documents:

Step 1: Find clusters closest to the query.

Step 2: Search only documents inside those clusters.

🚀 Why Faster?

Because: ✓ **Number of clusters** << **number of documents**

Searching clusters first greatly reduces computations.

⚖️ Tradeoff

This method is:

✓ faster

✗ slightly approximate

But clustering usually keeps related documents together, so **quality loss is small**.

book qs:

Exercise 16.1 Define two documents as similar if they have at least two proper names like Clinton or Sarkozy in common. Give an example of an information need and two documents, for which the cluster hypothesis does not hold for this notion of similarity.

cluster hypothesis says: Similar documents tend to be relevant to the same information needs.

We must find a case where:

✓ documents are “similar” by the given rule

✗ but NOT relevant to the same query

Doc 1: Barack Obama met Angela Merkel to discuss global climate agreements and renewable energy policies.

Doc 2: Barack Obama and Angela Merkel attended a ceremony celebrating diplomatic relations between the US and Germany.

> Both contain the proper names, so according to the similarity rule:

✓ They are in the same cluster.

✗ but they do NOT behave similarly regarding relevance

Therefore: **cluster hypothesis FAILS**

Exercise 16.2: Make up a simple, one-dimensional example (i.e., points on a line) with two clusters where the inexactness of cluster-based retrieval shows up. In your example, retrieving clusters close to the query should do worse than direct nearest neighbor search.

Cluster-based retrieval:

- 1 Finds nearest cluster
- 2 Searches only inside that cluster

Problem: ✗ Relevant document may lie in another cluster.

Consider two clusters on a line:

Cluster A: {1, 2, 3}

Cluster B: {8, 9, 10}

Let the query point be $q = 6$.

Direct nearest neighbor search examines all points and finds that point 8 is the closest (distance 2).

Suppose cluster-based retrieval searches only Cluster A due to an approximate cluster selection step. Then the closest retrieved point is 3 (distance 3), while the true nearest neighbor 8 is missed.

Thus, cluster-based retrieval performs worse than direct nearest neighbor search.

< samajh nahi aya >

🌟 [16.2] Problem Statement of Clustering

Given Things in Clustering

1. A Set of Documents

$$D = \{d_1, d_2, d_3, \dots, d_N\}$$

- (D) = collection of documents
- $(d_1, d_2, \dots)$ = individual documents
- (N) = total number of documents

2. Desired Number of Clusters

- $K \rightarrow$ This is the number of groups we want.

3. Objective Function

Mathematical function that **measures how good or bad a clustering is** (quality of clustering)

Goal of Clustering

We want to find an assignment:

$$\gamma : D \rightarrow \{1, 2, \dots, K\}$$

Meaning: **Each document is assigned to one cluster.**

Meaning of Gamma (γ)

Example

Document	Assigned Cluster
d_1	1
d_2	2
d_3	1

So: $\gamma(d_1)=1$

means document (d_1) belongs to cluster 1.

Main Goal

Find the assignment that:

✓ **minimizes** the **objective function**
(or sometimes maximizes it) → depending on the clustering method.

- γ should usually be **surjective**.

 Meaning: **Every cluster must contain at least one document.**

Similarity and Distance

Clustering quality is usually measured using:

- **Similarity** OR **distance** between documents.
-

K-Means Objective

 Minimize distance between:

- documents
- and
- cluster centroids

Centroid?

- center point (avg) of a cluster.

K-Means Goal

Minimize: distance(document, centroid)

OR equivalently: maximize similarity between documents and their centroids

Topic Similarity

Usually clustering tries to group documents by topic.

Example: China Documents

Documents about China may contain words like:

- Chinese
- Beijing
- Mao

So they become similar in vector space.

Example: UK Documents

Documents about UK may contain:

- London
- Britain
- Queen

Thus they form another cluster

English vs French Example

guage, then a different representation may be appropriate. When computing topic similarity, stop words can be safely ignored, but they are important cues for separating clusters of English (in which the occurs frequently and la infrequently) and French documents (in which the occurs infrequently and la frequently).

Stop words are usually **ignored** in topic clustering.

✗ But for Language Clustering...

Stop words become **VERY important**.

Hard Clustering vs Partitional Clustering

Hard Clustering

> Def A (**EXCLUSIVE**)

- Each doc belongs to **exactly one** cluster
- Membership values are only **0 or 1**
 - **either fully in cluster**
 - **or not in cluster**

> Def B (**ALTERNATIVE / MULTIPLE MEMBERSHIP**)

- A document can be a **full member of more than one cluster**
 - but the **membership values** are **binary**—either 0 (not a member) or 1 (full member)
-

Partitional Clustering

- Each document belongs to **exactly ONE** cluster.
- **No overlaps** allowed.

Note: In a *partitional hierarchical clustering*, a document belongs to a specific cluster and all its parent clusters

Hierarchical Exception

In hierarchical clustering, a document can belong to:

- its **own cluster**
- **parent cluster**

because clusters are **nested**.

Soft Clustering

- **Membership** can be **fractional**.
- Can take any non negative value

Example:

Cluster	Membership
Sports	0.7
Politics	0.3

Exhaustive Clustering

Every document must belong to some cluster.

Nonexhaustive Clustering

Some documents may belong to: **✗ no cluster**

Exclusive Clustering

Each document belongs to:

- **exactly one cluster** OR **no cluster**
-

[16.2.1] Cardinality – Number of Clusters

Cardinality here means: **Number of clusters (K)**

Why Choosing K is Difficult

Choosing the correct number of clusters is a major challenge.

Often:

- ✓ based on **experience**
- ✓ **domain knowledge**
- ✓ **heuristics**

- Applications can put **constraints** on the **range of K**

Example Constraint

In old scatter-gather systems: **Too many clusters could not fit on screen.**

So: **$K \approx 10$**

Clustering is a Search Problem

The algorithm **searches** for the **best partition of documents.**

✗ Brute Force Solution

Try ALL possible clusterings, then pick the best one.

Problem

- Number of possible clusterings grows **exponentially**.
 - So brute force is impossible for large datasets.
-

Iterative Refinement

Most clustering algorithms work **iteratively**.

- 1 start with an **initial clustering**
 - 2 **improve** it **step by step**
-

If the **starting point is poor**: ✗ algorithm **may miss** the **best clustering**

This is called **missing the: Global Optimum**

Global Optimum → **best possible solution overall**.

Therefore: **Good initialization** is very **important** in clustering.

Especially in K-Means.

★ [16.3] Evaluation of Clustering

A good clustering should have:

✓ **High intracluster similarity**

✓ **Low intercluster similarity**

📌 **Intracluster Similarity** → Documents **INSIDE** the **same cluster** should be **similar**.

📌 **Intercluster Similarity** → Documents from **DIFFERENT** clusters should be **dissimilar**.

These similarity-based measures are called: **Internal Criteria**

because they evaluate clustering using only:

- Distances, similarities, cluster structure

without external labels.

> But good scores on an internal criterion do not necessarily translate into good effectiveness

Direct Evaluation

- Alternative to internal criteria

Instead of only mathematical evaluation, we can **test clustering in the actual application.**

Example for search engines:

🕒 Measure how quickly users find answers.

> This is more realistic.

✗ Problem

Direct evaluation is:

- ⊘ **expensive**
 - ⊘ **time-consuming**
 - ⊘ **requires user studies**
-



Gold Standard

- Gold standard is a **correct reference classification made by humans.**
-



External Criteria

We **compare clustering against the gold standard.**

These are called: **External Criteria**

because **evaluation uses external human-labeled classes.**



Four External Evaluation Measures

- 1) Purity
 - 2) Normalized mutual information **NMI**
 - 3) Rand index
 - 4) F measure
-



1. Purity

Purity measures: **How “pure” each cluster is regarding class membership.**



How Purity Works

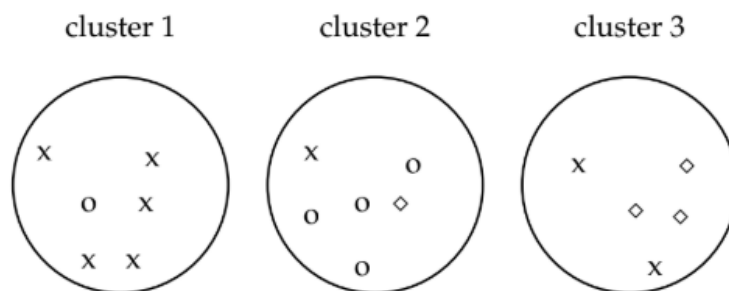
› each cluster is assigned to the class which is most frequent in the cluster.

For every cluster:

- 1 Find the majority class
- 2 Count how many docs belong to it
- 3 Sum these counts
- 4 Divide by total documents



Figure 16.4 Explanation



- To compute purity:
 - $5 = \max_j |\omega_1 \cap c_j|$ (class x, cluster 1);
 - $4 = \max_j |\omega_2 \cap c_j|$ (class o, cluster 2); and
 - $3 = \max_j |\omega_3 \cap c_j|$ (class ◊, cluster 3).
- Purity is $(1/17) \times (5 + 4 + 3) \approx 0.71$.



Purity Formula

$$\text{purity}(\Omega, \mathbb{C}) = \frac{1}{N} \sum_k \max_j |\omega_k \cap c_j|$$

- $\Omega = \{\omega_1, \omega_2, \dots, \omega_k\}$ is the set of clusters and $\mathbb{C} = \{c_1, c_2, \dots, c_j\}$ is the set of classes.
- For each cluster ω_k : find class c_j with most members n_{kj} in ω_k
- Sum all n_{kj} and divide by total number of points

Interpretation

Purity

Meaning

1

Perfect clustering

0

Very bad clustering

Problem with Purity

Purity becomes artificially **high** if we create **many clusters**.

Example:

If **every document becomes its own cluster**:

✓ **Purity = 1** → But clustering becomes meaningless.

★ 2. Normalized Mutual Information (NMI)

How much information clusters give us about the true classes.

If clusters perfectly match classes: ✓ knowing the cluster tells us the class.

If clusters are **random**: ✗ cluster gives **no useful information**.

📌 Mutual Information (MI)

Measures **shared information** between:

- **clusters**
- **true classes**

🎯 Important Property

Situation	MI
Random clustering	0
Perfect clustering	High

✗ Problem with MI

Like purity, **MI becomes large** if we **create many tiny clusters**.

Solution: Normalization

NMI divides MI by **entropy-based normalization**.

$$\text{NMI}(\Omega, \mathbb{C}) = \frac{I(\Omega; \mathbb{C})}{[H(\Omega) + H(\mathbb{C})]/2}.$$

› I is mutual information ⇒ [chp 13](#)

Entropy

Entropy **measures uncertainty/randomness**.

More clusters → **higher entropy**.

Normalization **prevents cheating by creating too many clusters**.

NMI Range

Value	Meaning
0	No relationship
1	Perfect match

🌟 3. Random Index (RI)

RI views clustering as **decisions about document pairs**.

For every pair: **Should they be together or separate?**

- No need for ground truth
- Alternative view to purity
- View it as a series of decisions
 - One for each of the $N(N - 1) / 2$ pairs of docs in collection.

📦 Four Possible Outcomes

✅ **True Positive (TP)**: Similar docs correctly placed in **SAME** cluster.

✅ **True Negative (TN)**: Different docs correctly placed in **DIFFERENT** clusters.

❌ **False Positive (FP)**: Different docs **incorrectly** placed together.

❌ **False Negative (FN)**: Similar docs **incorrectly** separated.

- We want to assign two docs to the same cluster if and only if they are similar.
- A **true-positive (TP) decision** assigns **two similar documents** to the **same cluster**
- a **true-negative (TN) decision** assigns **two dissimilar documents** to **different clusters**.

2 types of errors we can commit: **FP** and **FN**

- **false-positive (FP)** decision assigns **two dissimilar documents** to the **same cluster**.

- A **false-negative (FN)** decision assigns **two similar** documents to **different clusters**.



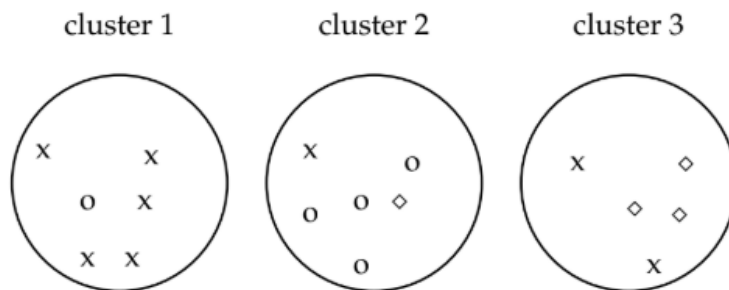
Rand Index Formula

Rand index (RI) measures the **RI percentage** of **decisions** that are **correct**.
(accuracy)

$$RI = \frac{TP + TN}{TP + FP + FN + TN}$$



Figure 16.4 calculation



Cluster	Number of Documents
Cluster 1	6
Cluster 2	6
Cluster 3	5

Total documents:



Master Table (MEMORIZE THIS)

True Reality	Clustering Decision	Result
Same class	Same cluster	✓ TP
Different class	Same cluster	✗ FP
Same class	Different clusters	✗ FN
Different class	Different clusters	✓ TN

To solve this problem, you need to consider this matrix:

TP: Same class + same cluster	FN: Same class + different clusters
FP: different class + same cluster	TN: different class + different clusters

[^ same as above]

- The symbols are the classes



Cluster 1

Contains:

x x x x x o

So:

- 5 x's
- 1 o

Total docs = 6



Cluster 2

Contains:

o o o o x ◇

So:

- 4 o's
- 1 x
- 1 diamond

Total = 6



Cluster 3

Contains:

◇ ◇ ◇ x x

So:

- 3 diamonds
- 2 x's

Total = 5

☀️ STEP 1: TP (True Positives)

TP means:

✅ same class AND ✅ same cluster

- So inside EACH cluster: **Count same-symbol pairs.**



Combination Formula

Whenever you see:

“How many pairs can I make from n items?”

use:

$$\binom{n}{2} = \frac{n(n-1)}{2}$$

$$TP = \binom{5}{2} + \binom{4}{2} + \binom{3}{2} + \binom{2}{2} = 20.$$

📁 Cluster 1 TP

There are 5 x's.

Number of x-x pairs:

$$\binom{5}{2} = 10$$

The single o cannot form same-class pair.

So:

TP from Cluster 1 = 10

📁 Cluster 2 TP

4 o's:

$$\binom{4}{2} = 6$$

Single x and single diamond cannot form pairs.

So:

TP from Cluster 2 = 6

Cluster 3 TP

3 diamonds:

$$\binom{3}{2} = 3$$

2 x's:

$$\binom{2}{2} = 1$$

Total:

$$3 + 1 = 4$$



Total TP

$$10 + 6 + 4 = 20$$



TP = 20

STEP 2: FP (False Positives) [2 ways to calc this]

We first count: **How many document pairs are in the SAME cluster?**

Because all such pairs are either:

- **TP** OR **FP**

Cluster 1

Contains 6 docs.

Number of pairs:

$$\binom{6}{2} = 15$$

Cluster 3

Contains 5 docs:

$$\binom{5}{2} = 10$$

Cluster 2

Also 6 docs:

$$\binom{6}{2} = 15$$

Total Same-Cluster Pairs

$$15 + 15 + 10 = 40$$

So:

$$TP + FP = 40$$

$$TP + FP = \binom{6}{2} + \binom{6}{2} + \binom{5}{2} = 40.$$

- We have **TP + FP**
- NOW, we **calculate TP** from the above method and **subtract TP** from TP + FP to get FP.

$$TP + FP = 40$$

and:

$$TP = 20$$

So:

$$FP = 40 - 20 = 20$$

OR

FP means: ✗ different classes BUT ✗ same cluster

So inside EACH cluster: **Count mixed-symbol pairs.**

Different classes but in the same cluster

$$\text{FP} = \underbrace{\binom{x_1}{1} \binom{o_1}{1}}_{c_1} + \underbrace{\binom{o_2}{1} \binom{x_2}{1} + \binom{o_2}{1} \binom{v_2}{1} + \binom{x_2}{1} \binom{v_2}{1}}_{c_2} + \underbrace{\binom{v_3}{1} \binom{x_3}{1}}_{c_3} = 5 + 4 + 4 + 1 + 6 = 20$$

Cluster 1 FP

Contains:

- 5 x
- 1 o

Mixed pairs:

Each x can pair with o.

So:

$$5 \times 1 = 5$$

Cluster 2 FP

Contains:

- 4 o
- 1 x
- 1 diamond

Mixed pairs:

$$\begin{pmatrix} 4 \\ 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

o-x:

$$4 \times 1 = 4$$

o-diamond:

$$4 \times 1 = 4$$

x-diamond:

$$1 \times 1 = 1$$

Total:

$$4+4+1=9$$

Cluster 3 FP

Contains:

- 3 diamonds
- 2 x

Mixed pairs:

$$3 \times 2 = 6$$

Total FP

$$5 + 9 + 6 = 20$$

$$\checkmark \text{ FP} = 20$$

☀ STEP 3: FN (False Negatives)

FN means:

✓ same class BUT ✗ different clusters

For each class: Find how many same-class pairs got split across clusters

$$\text{FN} = \begin{pmatrix} 5 \\ 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} + \begin{pmatrix} 5 \\ 1 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} \begin{pmatrix} 4 \\ 1 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} \begin{pmatrix} 3 \\ 1 \end{pmatrix} = 5 + 10 + 4 + 2 + 3 = 24$$

✨ X-class Distribution

x's are spread across:

Cluster	Count
C1	5
C2	1
C3	2

Now compute cross-cluster x-pairs:

Between C1 and C2:

$$5 \times 1 = 5 \quad \begin{pmatrix} 5 \\ 1 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

Between C1 and C3:

$$5 \times 2 = 10 \quad \begin{pmatrix} 5 \\ 1 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

Between C2 and C3:

$$1 \times 2 = 2 \quad \begin{pmatrix} 1 \\ 1 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix}$$

Total x FN:

$$5+10+2=17$$

✨ O-class Distribution

Cluster	Count
C1	1
C2	4

Cross-cluster pairs:

$$1 \times 4 = 4 \quad \begin{pmatrix} 1 \\ 1 \end{pmatrix} \begin{pmatrix} 4 \\ 1 \end{pmatrix}$$

💎 Diamond-class Distribution

Cluster	Count
C2	1
C3	3

Cross-cluster pairs:

$$1 \times 3 = 3$$

$$\begin{pmatrix} 1 \\ 1 \end{pmatrix} \begin{pmatrix} 3 \\ 1 \end{pmatrix}$$



Total FN

$$17 + 4 + 3 = 24$$



FN = 24

☀️ STEP 4: TN (True Negatives)

TN means: ❌ different classes AND ❌ different clusters

These are **correctly separated pairs**.

- Find **total possible pairs**
- **Subtract TP, FP, FN** from it to get TN

We already know:

Measure	Value
TP	20
FP	20
FN	24



Total Pairs Formula

Total documents:

17

Total possible pairs:

$$\binom{17}{2} = 136$$

So:

$$TN = 136 - 20 - 20 - 24 = 72$$

🌟 Final RI Calculation

$$RI = \frac{TP+TN}{TP+FP+FN+TN}$$

	same cluster	different clusters
same class	TP = 20	FN = 24
different classes	FP = 20	TN = 72

RI is then $(20 + 72)/(20 + 20 + 24 + 72) \approx 0.68$.

🎯 Interpretation

RI ≈ 0.68 means: **68% of pairwise decisions are correct.**

❌ Problem with RI

RI treats:

- FP errors
- FN errors

as **equally bad.**

But sometimes **separating similar docs is worse**

🌟 4. F-Measure

F-Measure F-measure combines:

- Precision
- Recall

and **allows different weighting of errors.**

$$P = \frac{TP}{TP + FP} \quad R = \frac{TP}{TP + FN} \quad F_{\beta} = \frac{(\beta^2 + 1)PR}{\beta^2 P + R}.$$

📌 Precision

Measures: Of retrieved same-cluster pairs, **how many were actually correct?**

📌 Recall

Measures: Of **all truly similar pairs**, **how many did we retrieve correctly?**

📊 Values

Using contingency table:

$$P = \frac{20}{40} = 0.5$$

$$R = \frac{20}{44} \approx 0.455$$

F-Measure Formula

$$F_{\beta} = \frac{(\beta^2 + 1)PR}{\beta^2 P + R}.$$

Meaning of β

β	Focus
$\beta = 1$	Equal weight
$\beta > 1$	More importance to Recall
$\beta < 1$	More importance to Precision

$F_1 \approx 0.48$ for $\beta = 1$ and $F_5 \approx 0.456$ for $\beta = 5$.

Measure	Value
F_1	≈ 0.48
F_5	≈ 0.46



Table 16.2 Summary

	purity	NMI	RI	F_5
lower bound	0.0	0.0	0.0	0.0
maximum	1.0	1.0	1.0	1.0
value for example	0.71	0.36	0.68	0.46

All four measures range from 0 (really bad clustering) to 1 (perfect clustering).

★ [16.4] K-Means Clustering

- K-means Clustering is a **flat clustering** algorithm.
- goal : **Put similar documents together into K clusters**

while making clusters:

- ✓ compact
- ✓ well-separated

- K-means tries to: **Minimize distance between points and their cluster center**

★ Cluster Center = Centroid

- **average position of all points in a cluster.**

$$\vec{\mu}(\omega) = \frac{1}{|\omega|} \sum_{\vec{x} \in \omega} \vec{x}.$$

Symbol	Meaning
ω	Cluster
$ \omega $	Number of points in cluster
x	A document/vector
$\mu(\omega)$	Centroid

- The **ideal cluster** in K-means is a **sphere** with the centroid as its center of gravity.
- clusters shouldn't overlap ❌

If **points** are **far from centroid**: ❌ cluster is bad

If **points** are **close**: ✅ cluster is good

★ RSS — Residual Sum of Squares

- K-means **minimizes RSS**.

RSS measures: **Total squared distance of all points from their centroids**

For one cluster:

$$RSS_k = \sum_{x \in \omega_k} |x - \mu(\omega_k)|^2$$

For all clusters:

$$RSS = \sum_{k=1}^K RSS_k$$

Points close to centroid

Small RSS ✓

Points far from centroid

Large RSS ✗

★ Why Squared Distance?

Because squaring: **penalizes far-away points more strongly**

★ K-Means Algorithm

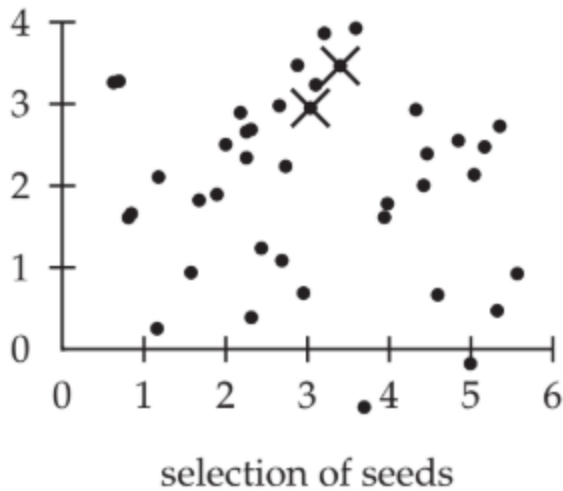
K-means works in:  **Repeat-until-converged style**

- Uses VSM to represent docs in feature dimensional space!!

★ STEP 1 — Choose Seeds

 **Seeds = Initial Centroids**

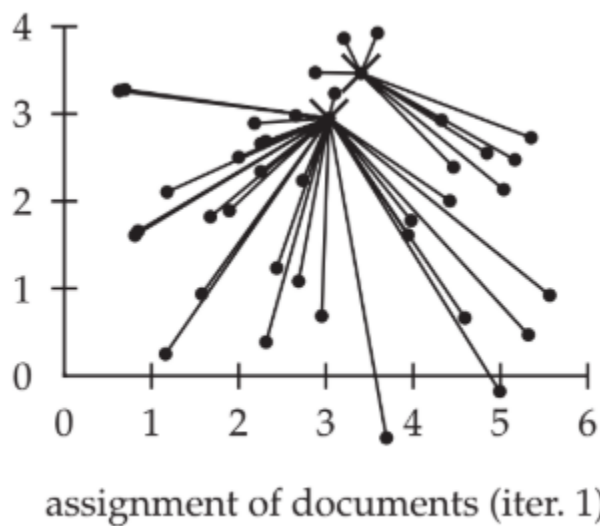
- **Randomly pick K starting points.**



➤ The X symbols are centroids.

☀ STEP 2 — Assign Points to Closest Centroid

- Every point **chooses nearest centroid**.
- This **creates clusters**.

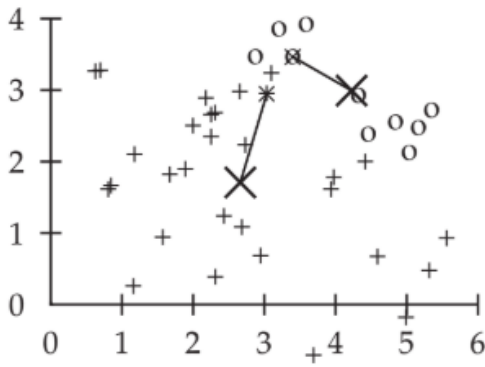


Each point belongs to: ☒ **Closest centroid only**

This is why K-means is: **Hard clustering**

☀ STEP 3 — Recompute Centroids

After assigning points: **Move centroid to average position of assigned points.**



recomputation/movement of $\bar{\mu}'$'s (iter. 1)

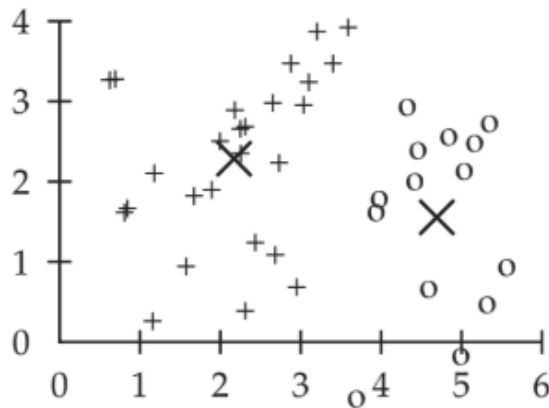
The X's **shift toward center of assigned points.**

☀ STEP 4 — Repeat

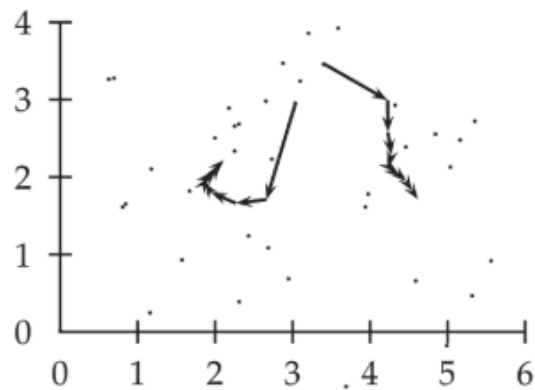
Repeat:

- 1 Assign points
- 2 Move centroids
- 3 Assign again
- 4 Move again

until stable.



$\vec{\mu}$'s after convergence (iter. 9)



movement of $\vec{\mu}$'s in 9 iterations

- Shows **centroid movement across 9 iterations**.
- Eventually **movement becomes tiny**.
- Clusters **stabilize**.
- **Centroids stop moving much**.
- This is called: **Convergence**

★ Pseudocode (Figure 16.5)

```

K-MEANS( $\{\vec{x}_1, \dots, \vec{x}_N\}, K$ )
1   $(\vec{s}_1, \vec{s}_2, \dots, \vec{s}_K) \leftarrow \text{SELECTRANDOMSEEDS}(\{\vec{x}_1, \dots, \vec{x}_N\}, K)$ 
2  for  $k \leftarrow 1$  to  $K$ 
3  do  $\vec{\mu}_k \leftarrow \vec{s}_k$ 
4  while stopping criterion has not been met
5  do for  $k \leftarrow 1$  to  $K$ 
6    do  $\omega_k \leftarrow \{\}$ 
7    for  $n \leftarrow 1$  to  $N$ 
8    do  $j \leftarrow \arg \min_{j'} |\vec{\mu}_{j'} - \vec{x}_n|$ 
9     $\omega_j \leftarrow \omega_j \cup \{\vec{x}_n\}$  (reassignment of vectors)
10   for  $k \leftarrow 1$  to  $K$ 
11   do  $\vec{\mu}_k \leftarrow \frac{1}{|\omega_k|} \sum_{\vec{x} \in \omega_k} \vec{x}$  (recomputation of centroids)
12  return  $\{\vec{\mu}_1, \dots, \vec{\mu}_K\}$ 

```

1. Line 1:

Randomly select k data points as initial seeds/starting centroids.

2. Lines 2–3:

Initialize each centroid μ_k with the selected random seed s_k .

3. Line 4:

Repeat the algorithm until convergence/stopping condition is met.

4. Lines 5–6:

Empty all clusters before reassignment begins.

5. Line 7:

Iterate through every data point in the dataset.

6. Line 8:

Compute the distance of the current point from all centroids and find the nearest centroid using:

$$j = \arg \min_{j'} |\mu_{j'} - x_n|$$

7. Line 9:

Assign the point to the nearest cluster ω_j .

8. Line 10:

After all points are assigned, iterate through all clusters again.

9. Line 11:

Recompute each centroid by taking the mean of all points in that cluster:

$$\mu_k = \frac{1}{|\omega_k|} \sum_{x \in \omega_k} x$$

10. Line 12:

Return the final clusters and centroids after convergence.

<some imp points frm book>

- Assignment of documents to clusters (the partitioning function γ) does not change between iterations. Except for cases with a bad local minimum, this produces a good clustering, but runtime may be unacceptably long.
- Centroids $\bar{\mu}_k$ do not change between iterations. This is equivalent to γ not changing (Exercise 16.5).

☀ Stopping/termination Conditions

3 STOPPING criteria for k-mean

1. few iteration

2. if no some clustering candidate want to change cluster

3. using threshold on intra compact on clusters

📌 1. Fixed Iterations

- condition **limits** the **runtime** of the clustering algorithm → **fast**
- **Poor clustering** cuz **stopped too early** → insufficient number of iterations

📌 2. RSS Threshold

Stop when RSS becomes **very small**.

Meaning: ☒ **clusters are compact enough**

Terminate when RSS falls below a threshold.

3. RSS Improvement Threshold

Stop when **RSS improvement becomes tiny**.

Example:

old RSS = 100, new RSS = 99.999

Almost no improvement → stop.

- **Terminate when the decrease in RSS falls below a threshold θ .**
-

Weaknesses of K-Means

 Sensitive to Initial Seeds

 Assumes Spherical Clusters

 Need to Choose K

 Can Get Stuck in Local Optimum

May find “good enough” solution but **not BEST possible solution**.

K-means always converges

- Eventually the algorithm stops changing clusters.
- K-means keeps **reducing: RSS (Residual Sum of Squares)** or leaves it **unchanged**.

- It **NEVER increases** RSS.
-

☀ Why Reassignment Step Decreases RSS

- **Reassignment** Step → Each **point chooses: nearest centroid**
- ❖ Suppose a point currently belongs to a far centroid.
 - If we move it to a closer centroid:
 - ☒ its distance decreases
 - Since RSS is sum of squared distances:
 - ☒ RSS decreases too.

☀ Why Recomputing Centroids Decreases RSS

📌 RSS for One Cluster

$$RSS_k(v) = \sum_{x \in \omega_k} |v - x|^2$$

🧠 Meaning

Suppose:

- cluster points are fixed
- we try different centroid positions v

decreases. Second, it decreases in the recomputation step because the new centroid is the vector $\bar{v}$ for which RSS_k reaches its minimum.

v is just:



"some possible center point"

The math is asking:

"If I place the cluster center at different locations, which location gives the smallest RSS?"

? Which v minimizes RSS?

Answer: The centroid (mean)

☀ Expanded Formula

$$RSS_k(v) = \sum_{x \in \omega_k} \sum_{m=1}^M (v_m - x_m)^2$$



Meaning of Symbols

Symbol	Meaning
M	Number of dimensions
v_m	m-th coordinate of centroid
x_m	m-th coordinate of point

☀ Partial Derivative Step

Apply differentiation to above formula!

$$\frac{\partial RSS_k(v)}{\partial v_m} = \sum_{x \in \omega_k} 2(v_m - x_m)$$

^ tells us how RSS changes if centroid moves slightly

Because in optimization: 📌 **Minimum occurs where derivative = 0**

☀ **Set Derivative = 0**

$$\sum_{x \in \omega_k} 2(v_m - x_m) = 0$$

Simplify:

$$\sum (v_m - x_m) = 0$$

Expand:

$$\sum v_m - \sum x_m = 0$$

Since v_m is constant:

$$|\omega_k| v_m = \sum x_m$$

Divide both sides:

$$v_m = \frac{1}{|\omega_k|} \sum x_m$$

🎯 **THIS IS THE CENTROID**

so,  **Mean minimizes RSS**

★ Why K-Means Eventually Stops

There are only **finite possible clusterings**

- A document **can only switch clusters finitely** many ways.
- Since RSS keeps decreasing: Eventually no better clustering exists.
- So algorithm converges.

Because there is only a **finite set of possible clusterings**, a **monotonically decreasing algorithm** will eventually arrive at a **(local) minimum**. Take care,

Monotonically decreases: either decreases or stays the same!

! Tie-Breaking Problem

- Suppose a document is **equally close to two centroids**.
- Then it may get stuck in a forever loop of clusterings that have the same cost

✓ Solution 📌 **Choose lowest-index cluster**

This prevents cycling.

★ Convergence $\neq$ Global Best Solution

- K-means converges.
 - BUT may converge to BAD solution called: **Local Optimum**
-

Type	Meaning
Global optimum	Best possible clustering
Local optimum	Good nearby solution only

! Why Does This Happen?

Because K-means depends heavily on: **Initial seeds**

Different seeds → different results.

★ Outliers Problem

An outlier is a **point very far from all others.**

- Suppose an **outlier becomes initial centroid.**
- Then: **No nearby points join it.**
- Result → **Singleton Cluster** (cluster containing only one document)
- This is usually bad clustering.

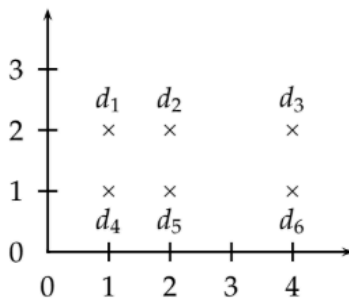


Figure 16.7 The outcome of clustering in K-means depends on the initial seeds. For seeds d_2 and d_5 , K-means converges to $\{\{d_1, d_2, d_3\}, \{d_4, d_5, d_6\}\}$, a suboptimal clustering. For seeds d_2 and d_3 , it converges to $\{\{d_1, d_2, d_4, d_5\}, \{d_3, d_6\}\}$, the global optimum for $K = 2$.

🌟 heuristics for seed selection

📌 1. Exclude Outliers

- Don't use isolated points as seeds.

📌 2. Try Multiple Random Starts

- Run K-means several times.
- Choose clustering with lowest RSS.

📌 3. Use Hierarchical Clustering First

- Use another clustering method to generate smarter seeds.
-

🌟 Clever Initialization Trick

- For each cluster: we pick i random points
- Average them and use their centroid as seeds for this cluster

This gives more stable seeds.

🌟 Time Complexity of K-Means

Symbol	Meaning
N	Number of documents
K	Number of clusters
M	Dimensions/features
I	Iterations

- Computing one distance of two vectors is $O(M)$.
- Reassignment step: $O(KNM)$ (we need to compute KN document-centroid distances)
- Recomputation step: $O(NM)$ (we need to add each of the document's $< M$ values to one of the centroids)
- Assume number of iterations bounded by I
- Overall complexity: $O(IKNM)$ – linear in all important dimensions
- However: This is not a real worst-case analysis.
- In pathological cases, complexity can be worse than linear.

K-means is **Linear** in:

- Iterations, clusters, Documents, dimensions

This makes it: ⚡ **Very efficient**

- K-means is **more efficient than** the **hierarchical** algorithms
 - K-means **quickly reaches** either **complete convergence** or a clustering that is **close to convergence**.
-

✗ problem

- Document vectors are **sparse**.
- Centroids are **dense**.

📌 Sparse Vector $\Rightarrow$ **Mostly zeros**.

📌 Dense Vector $\Rightarrow$ **Many nonzero values**.

Centroids **combine ALL words from cluster documents**.

So the **centroid becomes large and dense**.

✗ Result:

Distance computations become **slower**.

★ Optimization Trick

Keep only **top important centroid terms**.

Example: $k = 1000$

- Hardly decreases cluster quality
- speeds up similarity computation a lot.

^ Same efficiency problem is addressed by **K-medoids**

★ K-Medoids [📌 Alternative to K-Means]

- **computes medoids** instead of centroids **as cluster centers**.

medoid $\rightarrow$ **Actual document closest to centroid**

Medoids are real sparse documents, So **faster distance computations**

Difference Between K-Means and K-Medoids

Feature	K-Means	K-Medoids
Cluster center	Mean (average) of points	Actual data point from dataset
Center called	Centroid	Medoid
Sensitive to outliers?	Yes, highly sensitive	Less sensitive

[16.4.1] Cluster cardinality in K-means

This section discusses: **how do we decide the best number of clusters (K)**

 **First Naive Idea:**  **Choose K that gives minimum RSS**

$$RSS = \sum |x - \mu|^2$$

Lower RSS = better clustering.

 **Problem:**

RSS ALWAYS decreases when **K increases**.

 **Why?**

More clusters = points are **closer to centroids**.

Extreme case: **$K = N$** (each document becomes its own cluster)

Then: **distance to centroid = 0**

So: **$RSS=0$**

✗ **But This Is Useless**, If every document is its own cluster: there is no meaningful grouping.

So: 📌 **Lowest RSS does NOT mean best clustering.**

★ **Solution 1: Knee Method**

$RSS_{\min}(K)$ $\Rightarrow$ **Best RSS achievable using K clusters.**

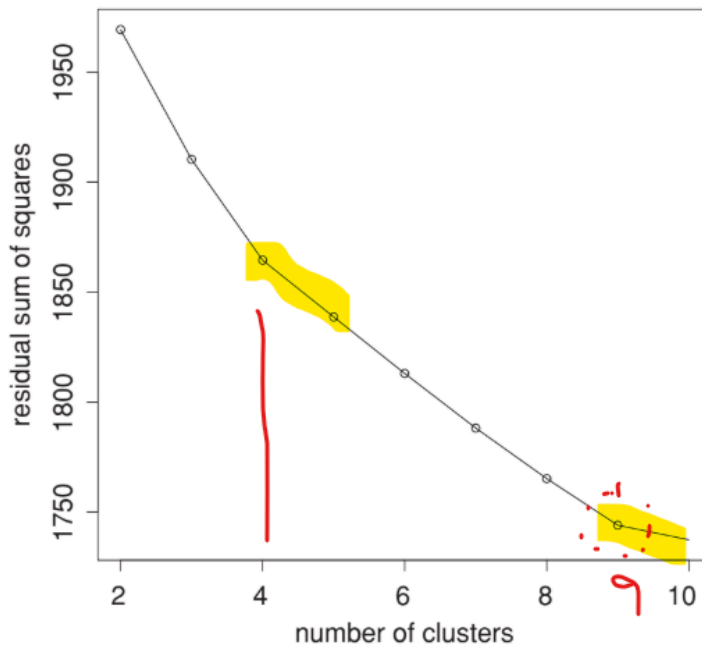
★ **Procedure**

For each K:

1. **Run K-means multiple times**
2. **Different random seeds each time**
3. **Take smallest RSS**

That becomes: **$\widehat{RSS}_{\min}(K)$.**

☀️ Then Plot a Graph



☀️ What Happens?

As K increases: RSS decreases.

BUT after some point, **improvement becomes much smaller.**

📌 This Bend Is Called 🧘 **Knee / Elbow Point**

That's usually a **good K.**

☀️ graph Explanation

The graph decreases sharply at first, then starts flattening.

📌 **Around K = 4,**

Adding more clusters after 4 doesn't improve RSS much.

So: ✅ **K = 4 is a good candidate**

★ Another Flattening at $K = 9$

This means **multiple reasonable clusterings may exist**

There is often **no single perfect K**

🌟 Second Idea: Penalize Too Many Clusters

Introduces **Distortion + Complexity Tradeoff**

Distortion how bad clustering is

Complexity how complicated model is

- Distortion measures how far points are from clusters.
- **distortion = RSS.**
- Complexity measures: **Number of clusters**
- **More clusters = more complex model.**

we minimize: **RSS + penalty for extra clusters**

$$K = \arg \min_K [RSS_{min}(K) + \lambda K]$$

- › Select K that minimizes $(RSS(K) + \lambda K)$

RSSmin(K) clustering error

λK penalty for too many clusters

☀️ λ (lambda): controls how strongly we punish extra clusters

- If λ is Small > Penalty is **weak** $\Rightarrow$ Algorithm **prefers MANY clusters**.
- If λ is Large > Penalty is **strong** $\Rightarrow$ Algorithm **prefers FEWER clusters**.
- Without penalty [$\lambda = 0$] algorithm always chooses $K = N$

> we can choose values of λ that have worked well for similar data sets in the past.

☀️ Akaike Information Criterion (AIC)

- **trades off distortion against model complexity.**

AIC balances:

Goal	Want
Fit data well	low distortion
Keep model simple	low complexity

$$\text{AIC: } K = \arg \min_K [-2L(K) + 2q(K)]$$

$L(K)$

how well model fits data

$q(K)$

model complexity

☀ AIC for K-Means

$$\text{AIC: } K = \arg \min_K [\text{RSS}_{\min}(K) + 2MK]$$

$M \Rightarrow$ number of dimensions/features [vocab size]

☀ Why Does 2MK Appear?

- Each centroid has: **M coordinates**
 - And there are: **K centroids**
 - So total parameters: **MK**
 - Complexity penalty becomes: **2MK**
-

✗ Problem in Text Clustering

Text data has **HUGE dimensions**.

Example: 50,000 terms

Then: **2MK becomes enormous**.

🧠 Result

Complexity term dominates RSS.

So AIC may incorrectly choose: $K = 1$ (one giant cluster)

☀ Conclusion

- Simple practical methods, like knee method, works better!!
-

book qs:

Exercise 16.4 Why are documents that do not use the same term for the concept “car” likely to end up in the same cluster in K-means clustering?

Documents that use different words for “car” can still end up in the same cluster because K-means **considers overall vector similarity, not just individual words**. Such documents often share many related terms and contextual features (e.g., engine, tires, road), making their vectors close in vector space. Over iterations, the **centroid of the cluster also captures related vocabulary**, causing semantically similar documents to cluster together.

Exercise 16.5 Two of the possible termination conditions for K-means were (i) assignment does not change, (ii) centroids do not change (page 332). Do these two conditions imply each other?

Condition	Meaning
(i) assignment does not change	documents stay in same clusters
(ii) centroids do not change	centroid positions stay same

If **document assignments do not change, then centroids also do not change** because centroids are computed as the mean of the same cluster members.

However, the reverse is not always true: **centroids may remain unchanged even if some documents change assignments**, especially in tie or symmetric cases.

Question No. 2

Consider the following data points in 2D space.

$P_1(3,1)$, $P_2(1,2)$, $P_3(1,1)$, $P_4(8,1)$, $P_5(7,3)$, $P_6(6,5)$ taking P_2 and P_5 as initial seeds – run vanilla k-means algorithm for two iterations to find the resultant clusters C_1 and C_2 .

First Iteration:

$$C_1 = P_2(1,2)$$

$$C_2 = P_5(7,3)$$

P_1 is close to P_2 than P_5 , similarly P_3 is close to P_2 than P_5
 P_4 is close to P_5 than P_2 . Hence

$$C_1 = \{P_1, P_2, P_3\} = \{5/3, 4/3\}$$

$$C_2 = \{P_4, P_5, P_6\} = (7, 3)$$

First Iteration:

$$C_1(5/3, 4/3)$$

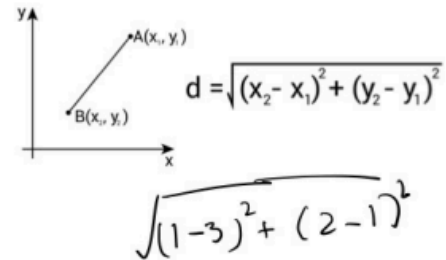
$$C_2 = (7, 3)$$

P_1 is close to C_1 than C_2 , similarly P_3 is close to C_1 than C_2
 P_4 is close to P_5 than P_2 . Hence

$$C_1 = \{P_1, P_2, P_3\} = \{5/3, 4/3\}$$

$$C_2 = \{P_4, P_5, P_6\} = (7, 3)$$

Distance Formula



$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

$$\sqrt{(1-3)^2 + (2-1)^2}$$